



Government
of Canada

Gouvernement
du Canada

ESDC Development Community

Git Branching, Picking a Git Flow

Le texte français est donné à la suite.

Branching

Managing your branching and versioning effectively for your project development is a key aspect to success when working with Git. Most git projects depend and follow a Branching Model that defines the rules and expectations from developers to manage their branches. The Branching Model helps provide a common practice that everyone can openly know and be able to work with.

Picking a Branching Model

Picking the right branching model for your project can be difficult because there are a lot of models out there and they all advocate they are the *best*. When picking a model look at your project first. Define some requirements that you need your model to meet based on your release cycles to all the different environments you support and what kind of project it is. An API or Web Service might need a different model than a Web App.

ESDC Projects Requirements

We have extracted a few requirements from our typical ESDC release process.

- Can manage a long term production equivalent branch separated from the day-to-day development.

Why? Our production release are sometimes 6 months or more apart.

- Can manage a semi-long term staging branch separated from the day-to-day development and production code.

Why? We have Testing and QA cycles that last about 2 months.

- Can handle emergency releases in a way that doesn't alter from the regular branching.

Why? ERs can be high stress situations, we don't want to change the "regular" method for moving code from branch to branch.

Popular Flows Comparison

Key Features	GitFlow	OneFlow	Microsoft Release Flow	GitHub Flow	GitLab Flow
Production and Development branches segregated *	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:
:grey_question: (Optional)	:heavy_check_mark:	:heavy_minus_sign:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:
Manages Staging/Release branches *	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:	:heavy_check_mark:
ER branching similar to "regular" *	:heavy_check_mark:	:heavy_check_mark:	:heavy_minus_sign:	:heavy_check_mark:	:heavy_check_mark:
Easy to follow history **	:heavy_minus_sign:	:heavy_check_mark:	:heavy_minus_sign:	:heavy_check_mark:	:heavy_check_mark:
Easy to learn when new to Git **	:heavy_check_mark:	:heavy_minus_sign:	:heavy_minus_sign:	:heavy_check_mark:	:heavy_check_mark:

:heavy_minus_sign:		:heavy_check_mark:		:heavy_minus_sign:		
Maintains all main branches (reduced chances of code loss)						
:heavy_check_mark:		:heavy_check_mark:		:heavy_minus_sign:		
:heavy_check_mark:		:heavy_check_mark:			Designed for Continuous	
Deployment				:heavy_minus_sign:		:heavy_minus_sign:
:heavy_minus_sign:		:heavy_check_mark:		:heavy_minus_sign:		
Designed for Continuous Delivery				:heavy_minus_sign:		
:heavy_minus_sign:		:heavy_check_mark:		:heavy_minus_sign:		
:heavy_check_mark:			Can handle Continuous Delivery			
:heavy_check_mark:		:heavy_check_mark:		:heavy_check_mark:		
:heavy_check_mark:		:heavy_check_mark:			Can handle Continuous	
Integration				:heavy_check_mark:		:heavy_check_mark:
:heavy_check_mark:		:heavy_check_mark:		:heavy_check_mark:		
Deploys to Production from "Main/Master/Default" branch						
:heavy_check_mark:		:heavy_check_mark:		:heavy_check_mark:		
:heavy_minus_sign:		:heavy_check_mark:				

Legend: * from **ESDC Projects Requirements**; ** Subjective analysis.

Our Recommendation

Use GitFlow branching model. It is great for managing releases and parallel development in large applications.

This model has two *primary* branches, `master` & `dev`, that always exist. All other branches only exist as long as they are active (until merged into a *primary* or *secondary* branch). The *secondary* branches include *release*, *feature* and *hotfix*. Both the *primary* and *secondary* branches should be

protected to only accept changes via a pull or merge request. The last branch is a *working* branch. This branch is the only place you should be committing code against.

GitFlow can be fairly simplistic but also get very complex. We recommend you try to keep it as simple as possible, but don't shy away from getting complex if that is what the project requires.

Texte français:

Ramification (Branching)

Gérer efficacement vos branches et vos versions pour le développement de vos projets est un facteur clé de réussite lors de l'utilisation de Git. La plupart des projets Git s'appuient sur un modèle de ramification (*Branching Model*) qui définit les règles et les attentes des développeurs pour gérer leurs branches. Ce modèle fournit une pratique commune, partagée et accessible à tous.

Choisir un modèle de ramification

Choisir le bon modèle de ramification pour votre projet peut s'avérer difficile en raison de la multitude de modèles existants, chacun prétendant être le *meilleur*. Pour choisir un modèle, examinez d'abord les besoins de votre projet. Définissez les exigences auxquelles votre modèle doit répondre en fonction de vos cycles de livraison vers les différents environnements pris en charge et du type de projet. Une API ou un service Web peut nécessiter un modèle différent de celui d'une application Web.

Exigences des projets d'EDSC

Nous avons dégagé quelques exigences issues du processus de livraison typique d'EDSC.

- Pouvoir gérer une branche équivalente à la production à long terme, distincte du développement quotidien.

Pourquoi ? Nos livraisons en production sont parfois espacées de 6 mois ou plus.

- Pouvoir gérer une branche de préproduction (staging) à moyen terme, distincte du développement quotidien et du code de production.

Pourquoi ? Nous avons des cycles de test et d'assurance qualité qui durent environ 2 mois.

- Pouvoir gérer les livraisons d'urgence (hotfix) sans déroger au schéma de ramification standard.

Pourquoi ? Les livraisons d'urgence peuvent être stressantes; nous ne voulons pas modifier la méthode standard de transfert de code d'une branche à l'autre.

Comparaison des flux populaires

| Fonctionnalités clés | [GitFlow](#) | [OneFlow](#) | [Microsoft Release Flow](#) |
[GitHub Flow](#) | [GitLab Flow](#) | | :--- | --- | --- | --- | --- | --- | --- | | Branches
de production et de développement séparées * | :heavy_check_mark: |
:grey_question: (Optionnel) | :heavy_check_mark: | :heavy_minus_sign: |
:heavy_check_mark: | | Gestion des branches de staging / release * |
:heavy_check_mark: | :heavy_check_mark: | :heavy_minus_sign: |
:heavy_minus_sign: | :heavy_check_mark: | | Ramification d'urgence

similaire au flux régulier * | :heavy_check_mark: | :heavy_check_mark: |
 :heavy_minus_sign: | :heavy_check_mark: | :heavy_minus_sign: | |
 Historique facile à suivre ** | :heavy_minus_sign: | :heavy_check_mark: |
 :heavy_minus_sign: | :heavy_check_mark: | :heavy_minus_sign: | | Facile à
 apprendre pour les débutants Git ** | :heavy_check_mark: |
 :heavy_minus_sign: | :heavy_minus_sign: | :heavy_check_mark: |
 :heavy_minus_sign: | | Maintient toutes les branches principales (risque
 réduit de perte de code) | :heavy_check_mark: | :heavy_check_mark: |
 :heavy_minus_sign: | :heavy_check_mark: | :heavy_check_mark: | | Conçu
 pour le déploiement continu | :heavy_minus_sign: | :heavy_minus_sign: |
 :heavy_minus_sign: | :heavy_check_mark: | :heavy_minus_sign: | | Conçu
 pour la livraison continue | :heavy_minus_sign: | :heavy_minus_sign: |
 :heavy_check_mark: | :heavy_minus_sign: | :heavy_check_mark: | | Prend
 en charge la livraison continue | :heavy_check_mark: | :heavy_check_mark: |
 | :heavy_check_mark: | :heavy_check_mark: | :heavy_check_mark: | | Prend
 en charge l'intégration continue | :heavy_check_mark: |
 :heavy_check_mark: | :heavy_check_mark: | :heavy_check_mark: |
 :heavy_check_mark: | | Déploie en production depuis la branche «
 Main/Master/Default » | :heavy_check_mark: | :heavy_check_mark: |
 :heavy_check_mark: | :heavy_minus_sign: | :heavy_check_mark: |

Légende : * tiré des **Exigences des projets d'EDSC**; ** Analyse
 subjective.

Notre recommandation

Utilisez le modèle de ramification [GitFlow](#). Il est idéal pour la gestion des
 livraisons et le développement parallèle dans les grandes applications.

Ce modèle comporte deux branches *principales*, `master` et `dev`, qui existent en permanence. Toutes les autres branches n'existent que le temps de leur utilisation active (jusqu'à ce qu'elles soient fusionnées dans une branche *principale* ou *secondaire*). Les branches *secondaires* comprennent *release*, *feature* et *hotfix*. Les branches principales et secondaires doivent toutes être protégées afin de n'accepter des modifications que par le biais d'une pull/merge request. La dernière branche est une branche de *travail* (working branch). C'est sur cette branche uniquement que vous devez valider votre code.

GitFlow peut paraître simple au départ mais peut également devenir très élaboré. Nous vous recommandons de le garder aussi simple que possible, sans hésiter à ajouter de la complexité si le projet le requiert.